

RAG и Агенты

Андросов Дмитрий, 04.05.2026, AI Masters

Введение

Задача информационного поиска

- Дано:
 - Набор документов $D = \{d_1, d_2, \dots, d_n\}$
 - Запрос $Q = \{q_1, q_2, \dots, q_m\}$
- Найти:
 - Набор документов $D^* \subset D$, релевантных запросу Q

Задача информационного поиска

- Дано:

- Набор документов $D = \{d_1, d_2, \dots, d_n\}$
- Запрос $Q = \{q_1, q_2, \dots, q_m\}$

- Найти:

- ~~• Набор документов $D^* \subseteq D$, релевантных запросу Q~~
- Достоверный ответ на запрос Q , опираясь на документы D

Наивный подход

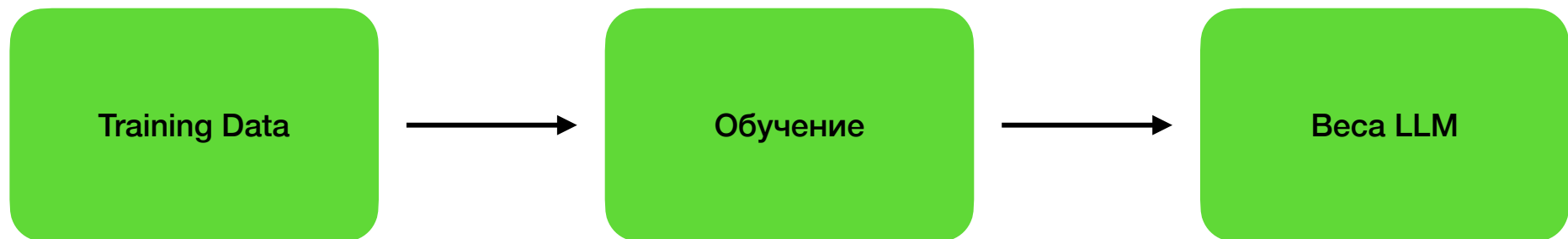
- Возьмем все знания о мире
- Научим LLM на этих знаниях
- Получим модель, имеющую достаточные знания для того, чтобы иметь ответ на практически любой вопрос



От LLM к RAG: зачем нам внешняя память

LLM с точки зрения информационного поиска

- LLM — это сжатая параметрическая память о текстах из обучающей выборки
- Модель "знает" информацию, но не "помнит" источник
- Знания вшиты в веса → нельзя обновить без дообучения
- Аналогия: эрудированный человек, прочитавший миллионы книг, но без доступа к библиотеке прямо сейчас
- Преимущества: обобщение, связная генерация, рассуждение



Проблема №1: Knowledge cutoff

- У каждой модели есть **дата отсечения знаний (cutoff date)** — момент, после которого данные не попали в обучение
- Между cutoff и моментом, когда вы задаёте вопрос, может пройти от месяцев до лет
- Реальный лаг: pre-training + alignment + тестирование + релиз = 6 – 12 месяцев минимум
- Мир меняется: новые законы, новые версии библиотек, новые события
- Для задач, где важна свежесть данных, параметрической памяти недостаточно по определению



Проблема №2: Галлюцинации

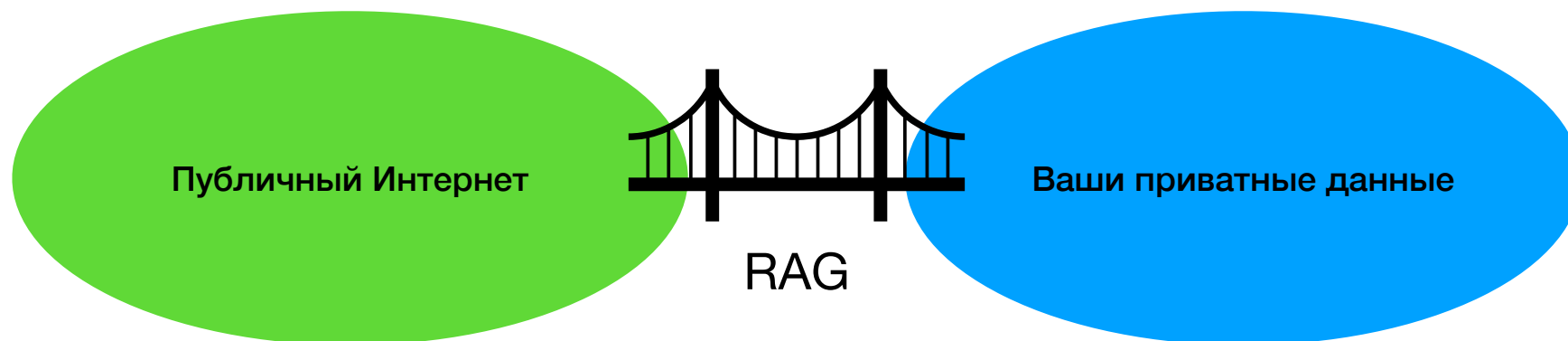
- LLM обучены генерировать правдоподобный текст, а не только истинный
- При недостатке знаний модель "достраивает" ответ — генерирует то, что похоже на правду, но не есть правда
- Особенно опасны: даты, имена, цифры, ссылки на статьи и законы — то, что модель не может вывести из контекста
- Чем увереннее звучит ответ, тем сложнее заметить галлюцинацию
- Примеры:
 - Программирование: вызов несуществующих функций библиотеки с правдоподобными именами
 - Академические ссылки: модель уверенно выдумывает названия статей, авторов

Проблема №3: Приватные и доменные данные

- LLM обучены на публичных данных интернета — Wikipedia, GitHub, книги
- В обучающей выборке нет: внутренней документации компании, базы знаний поддержки, корпоративной wiki, переписок, внутренних регламентов
- Дообучить на своих данных — дорого, требует перетренировки при каждом обновлении базы и не решает проблему свежести
- Для многих организаций отправка данных в обучение третьей стороны вообще запрещена политиками безопасности или законодательством

Проблема №3: Приватные и доменные данные

- Конкретные сценарии, где RAG необходим:
 - Чат-бот техподдержки по внутренним регламентам и FAQ
 - Поиск по корпоративной базе знаний (Confluence, корпоративный портал)
 - Ассистент юриста по базе договоров организации
 - Медицинский ассистент по протоколам конкретной клиники
 - Поиск по документации продукта



Проблема №4: Верифицируемость

- Ответ LLM приходит как "монолитный текст" — без указания, откуда взят каждый факт
- Пользователь не может проверить ответ, кроме как загуглить заново
- Для критичных доменов (медицина, право, финансы) это блокер — нужны ссылки на источники
- Регуляторные требования: в ряде индустрий запрещено использовать "чёрные ящики" для принятия решений

Проблема №4: Верифицируемость

- Что дает RAG:
 - Каждый факт можно сопоставить с конкретным фрагментом конкретного документа
 - Цитирование источников (как делает Perplexity или поиск с ИИ от Яндекса/Google)
 - Audit trail: можно постфактум разобрать, почему система ответила именно так
 - Возможность пользователю провалиться в первоисточник одним кликом

RAG (Retrieval-Augmented Generation) — метод ИИ, сочетающий поиск информации из внешних источников (например, баз данных или документов) с генерацией текста. Модель сначала извлекает релевантные данные, затем использует их для формирования точного и обоснованного ответа, снижая риск выдачи непроверенной информации. Применяется в чат-ботах, системах поддержки и других сценариях, где важна актуальность данных.



RAG (Retrieval-Augmented Generation, «генерация с дополненной выборкой») — это подход, при котором языковая модель перед ответом сначала ищет нужную информацию во внешней базе знаний (например, документы, вики, корпоративные справочники), а уже затем генерирует ответ на основе этого найденного контекста. [yandex +2](#)

Очень просто

- Есть языковая модель (LLM) и отдельная база данных / документов. [astera +1](#)
- На запрос пользователя система сначала ищет релевантные фрагменты текста, затем подставляет их в промпт и просит LLM ответить, опираясь на эти данные. [habr +1](#)

Так RAG снижает риск галлюцинаций и позволяет использовать актуальные или доменные знания без переобучения самой модели. [simpleone +1](#)

10 источников



Альтернатива №1

Fine-tuning. Почему этого недостаточно

- Fine-tuning — дообучение модели на ваших данных
- Что fine-tuning делает хорошо: меняет стиль, формат ответов, учит специфическим задачам, форматированию выхода
- Что fine-tuning делает плохо для задачи "помни факты":
 - Дорого по compute и инженерным ресурсам
 - Каждое обновление базы → дообучение и редеплой
 - Нет гарантий, что конкретный факт запомнится — обучение это статистический процесс
 - По-прежнему нет ссылок на источники
 - По-прежнему возможны галлюцинации
 - Не решает проблему приватности — данные нужно подавать в обучение

Альтернатива №1

Fine-tuning. Почему этого недостаточно

Критерий	Fine-tuning	RAG
Свежесть данных	Плохо (нужна перетренировка)	Отлично (обновление индекса)
Стоимость обновления	Высокая	Низкая
Ссылки на источники	Нет	Да
Защита от галлюцинаций	Слабая	Сильная
Изменение стиля/формата	Отлично	Слабо
Сложность инфраструктуры	Высокая	Средняя

Альтернатива №2

Длинный контекст. Почему этого недостаточно

- Современные модели поддерживают огромные окна контекста: Claude — 200k, Gemini — 1-2M, YandexGPT 5 — 32k токенов
- Соблазнительная идея: давайте просто положим всю базу знаний в контекст

Альтернатива №2

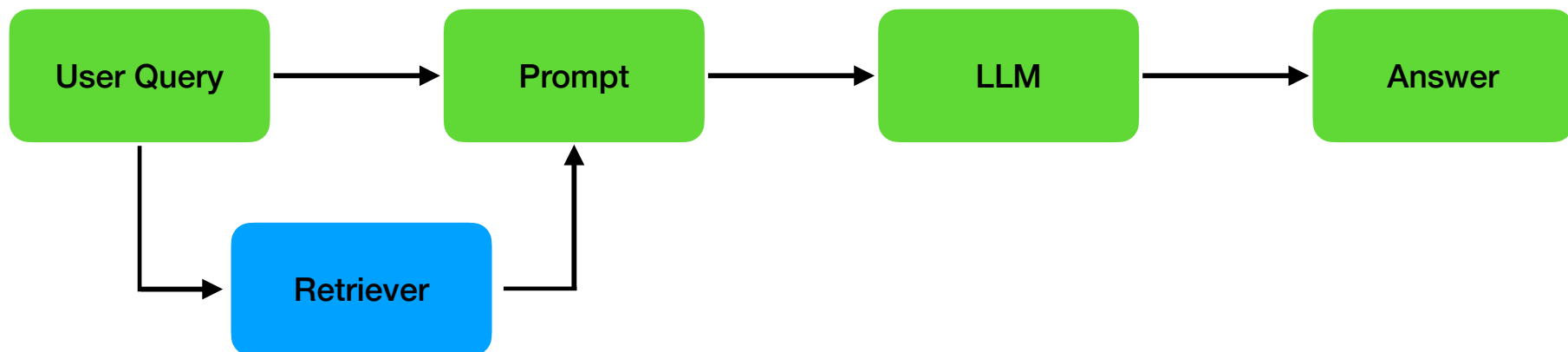
Длинный контекст. Почему этого недостаточно

- Ограничения длинного контекста:
 - **Стоимость:** оплата за входные токены, при 1M токенов на каждый запрос экономика разрушается
 - **Латентность:** даже на современном железе обработка 1M токенов это секунды-десятки секунд
 - **Lost in the middle:** модели плохо используют информацию из середины длинного контекста, лучше всего — из начала и конца
 - **Физический предел:** даже 2M токенов это ~1.5 млн слов = ~5000 страниц. Корпоративная база — десятки и сотни тысяч страниц
 - **Безопасность и доступ:** нельзя положить в контекст документы, к которым у пользователя нет доступа

Идея RAG

Гибрид параметрической и непараметрической памяти

- **Параметрическая память** (веса LLM): общие знания о языке, мире, рассуждения, стиль
- **Непараметрическая память** (внешний индекс): актуальные факты, приватные данные, специфика домена
- **RAG** = на каждый запрос найти релевантные фрагменты во внешней памяти и передать их в LLM как контекст



Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего

Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего
- **2019** — REALM (Google): первая попытка соединить retrieval с pretraining'ом языковой модели

Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего
- **2019** — REALM (Google): первая попытка соединить retrieval с pretraining'ом языковой модели
- **Май 2020** — Facebook AI: статья "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — здесь впервые появляется термин RAG в современном понимании

Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего
- **2019** — REALM (Google): первая попытка соединить retrieval с pretraining'ом языковой модели
- **Май 2020** — Facebook AI: статья "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — здесь впервые появляется термин RAG в современном понимании
- **2022-2023** — взрывной рост после появления ChatGPT: LangChain (октябрь 2022), LlamaIndex (ноябрь 2022), массовое внедрение в индустрии

Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего
- **2019** — REALM (Google): первая попытка соединить retrieval с pretraining'ом языковой модели
- **Май 2020** — Facebook AI: статья "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — здесь впервые появляется термин RAG в современном понимании
- **2022-2023** — взрывной рост после появления ChatGPT: LangChain (октябрь 2022), LlamaIndex (ноябрь 2022), массовое внедрение в индустрии
- **2024** — продвинутые техники: Self-RAG, Corrective RAG, GraphRAG (Microsoft, апрель 2024), Contextual Retrieval (Anthropic, сентябрь 2024)

Откуда пришел RAG

- **2017** — Attention Is All You Need: база для всего последующего
- **2019** — REALM (Google): первая попытка соединить retrieval с pretraining'ом языковой модели
- **Май 2020** — Facebook AI: статья "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — здесь впервые появляется термин RAG в современном понимании
- **2022-2023** — взрывной рост после появления ChatGPT: LangChain (октябрь 2022), LlamaIndex (ноябрь 2022), массовое внедрение в индустрии
- **2024** — продвинутые техники: Self-RAG, Corrective RAG, GraphRAG (Microsoft, апрель 2024), Contextual Retrieval (Anthropic, сентябрь 2024)
- **2025-2026** — переход к агентным системам: Deep Research (OpenAI), повсеместное распространение agentic RAG

План лекции

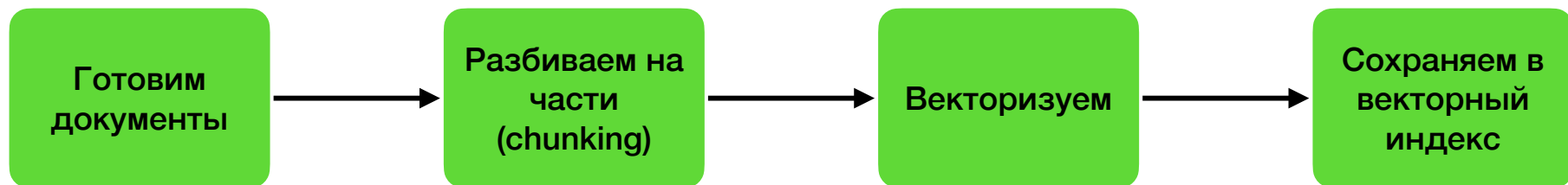
- Naïve RAG: базовая архитектура
- Chunking и подготовка данных
- Продвинутое техники retrieval-a
- Оценка качества RAG-систем
- От RAG к агентам

Naive RAG: базовая архитектура

Два этапа жизни RAG-системы

Indexing

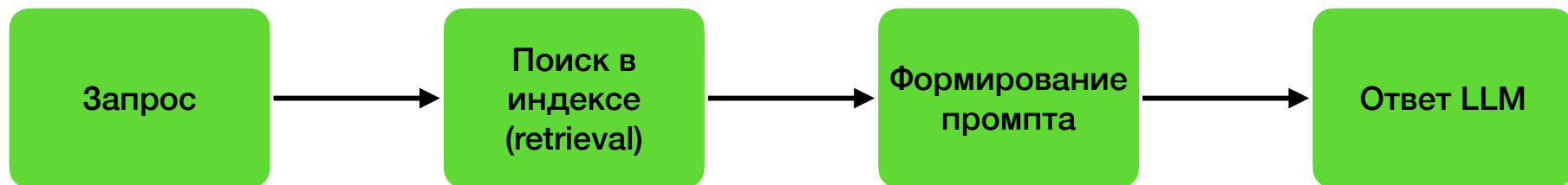
- Происходит заранее или по расписанию
- Делается редко (раз в день/неделю или при обновлении данных)
- Может занимать часы для больших баз



Два этапа жизни RAG-системы

Querying

- Происходит в реальном времени на каждый запрос пользователя
- Должно быть быстрым (секунды)
- Тысячи раз в секунду в нагруженной системе



Indexing pipeline

Шаг 1: Загрузка и парсинг документов

- Собираем сырые данные из источников
- Источники бывают очень разные: PDF (отчёты, договоры), DOCX (внутренняя документация), HTML (веб-страницы, Confluence), Markdown (тех. документация, README), JSON/XML (API), базы данных, email, чаты
- Каждый формат требует своего парсера
- Главные сложности: PDF (особенно сканы), таблицы, изображения, формулы

Indexing pipeline

Шаг 2: Chunking

- Разбиение документов на фрагменты - chunk-и
- Зачем разбивать:
 - Эмбеddинг-модели имеют ограничение на длину входа (обычно 512-8192 токена)
 - Векторизовать целый документ = "размывать" семантику в одном векторе
 - В контекст LLM нужно передать фрагменты, а не целые документы (стоимость, lost-in-the-middle)

Indexing pipeline

Шаг 3: Embeddings

- Каждый chunk превращаем в плотный вектор фиксированной размерности
- Используем embedding-модель — обычно encoder-only трансформер (часто bi-encoder — из прошлой лекции)
- Размерность — обычно 384, 768, 1024, 1536, 3072
- Модель должна быть той же самой на этапах indexing и querying (иначе векторы несравнимы)

Indexing pipeline

Шаг 4: Vector Storage

- Сохраняем векторы вместе с метаданными в специализированное хранилище
- Vector store умеет:
 - Хранить миллионы/миллиарды векторов
 - Быстро находить ближайших соседей по cosine similarity / dot product / L2
 - Использовать ANN-индексы (HNSW, Faiss) для скорости — ценой небольшой потери точности
 - Фильтровать по метаданным (например, "только документы за 2025 год")

Indexing pipeline

Шаг 4: Vector Storage — что хранится для каждого chunk-а

```
{
  "id": "doc_42_chunk_7",
  "vector": [0.012, -0.034, ..., 0.218],
  "text": "Согласно ст. 152 НК РФ, ставка НДС...",
  "metadata": {
    "source": "tax_code_2025.pdf",
    "page": 47,
    "section": "Глава 21",
    "date": "2025-01-01",
    "access_level": "public"
  }
}
```

Querying pipeline

- **Шаг 1:** Получаем запрос: "Какова ставка НДС для медицинских товаров?»
- **Шаг 2:** Векторизуем запрос той же embedding-моделью
- **Шаг 3:** Ищем top-k ближайших чанков в vector store
- **Шаг 4:** Формируем промпт для LLM

System: Ты ассистент, отвечающий на вопросы по налоговому праву РФ.
Используй ТОЛЬКО предоставленный контекст. Если ответа в контексте нет — скажи "не знаю". Указывай источники.

Context:

[1] (tax_code_2025.pdf, стр. 47): Согласно ст. 164 НК РФ, ставка 10% применяется...

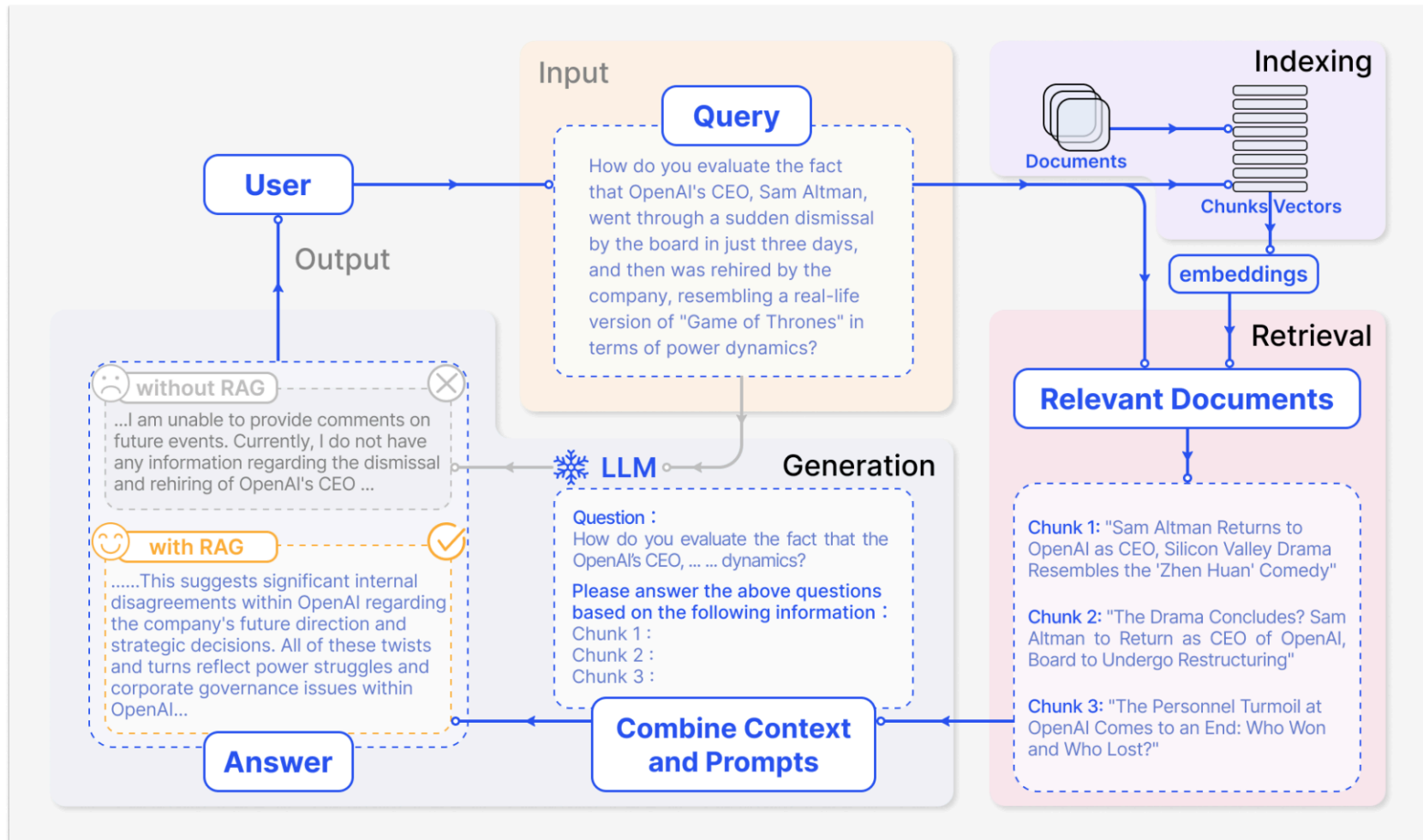
[2] (tax_code_2025.pdf, стр. 49): К медицинским товарам относятся...

[3] ...

Question: Какова ставка НДС для медицинских товаров?

- **Шаг 5:** Отправляем в LLM, получаем ответ

Naive RAG



Преимущества Naïve RAG

- **Свежие данные:** добавили документ в индекс — он сразу доступен, без ретренинга
- **Прозрачность:** каждый ответ можно сопоставить с исходными чанками
- **Цитирование:** легко добавить ссылки на источники в ответ
- **Контроль приватности:** индекс на ваших серверах, доступ к данным контролируемый
- **Дёшево относительно fine-tuning:** один embedding-проход на документ, дальше только хранение и поиск
- **Снижение галлюцинаций:** модель опирается на контекст, а не на параметрическую память
- **Низкий порог входа:** неделя на реализацию MVP

Проблемы Naïve RAG

Проблемы chunking

- Разрывает смысловые единицы
- Чанк теряет контекст всего документа
- Таблицы, формулы, код — особые случаи

Проблемы Naive RAG

Проблемы retrieval

- Vocabulary mismatch: запрос и документ говорят о том же разными словами
- Сложные запросы: "сравни X и Y" — нужно несколько разных retrieval'ов
- Плохое ранжирование: cosine similarity $\neq$ релевантность
- Нерелевантные результаты в top-k: засоряют контекст, путают модель

Проблемы Naive RAG

Проблемы generation

- Модель может проигнорировать контекст и галлюцинировать всё равно
- Lost in the middle: важный чанк в середине промпта может быть проигнорирован
- Модель может скомбинировать факты из разных чанков неправильно

Проблемы Naïve RAG

Проблемы оценки

- Как понять, что система работает хорошо?
- Что считать правильным ответом?
- Где упало качество — в retrieval или в generation?

Проблемы Naïve RAG

Архитектурные проблемы

- Один retrieval-call на запрос — иногда нужно несколько
- Не все запросы требуют retrieval-a

Пример провала Naïve RAG

- Запрос пользователя: «Сравни условия ипотеки в Сбербанке и ВТБ для семей с детьми»
- Что найдёт naïve RAG в top-5:
 - 2 чанка про общие условия ипотеки в Сбере
 - 1 чанк про вклады в ВТБ
 - 1 чанк из википедии про историю ипотеки
 - 1 чанк про семейную ипотеку, но без указания банка

Пример провала Naïve RAG

Почему это плохо

- Запрос неявно состоит из двух подзапросов (про Сбер и про ВТБ) — нужно было сделать два retrieval-a
- "семьи с детьми" — это семантически "семейная ипотека", но модель эмбедингов могла этого не уловить
- Возможна асимметрия: про Сбер документов больше, ВТБ "недопредставлен" в выдаче
- Модель сгенерирует ответ, в котором про ВТБ почти ничего не будет, но прозвучит это уверенно

Пример провала Naïve RAG

Что нужно для правильного ответа

- Декомпозиция запроса (разобрать сложный вопрос на части)
- Hybrid search (BM25 + dense)
- Reranking, чтобы отсеять нерелевантное
- Возможно, agentic подход с несколькими retrieval-ами
- Метрики, чтобы понять, что система ошиблась

Naive RAG

Что мы узнали

- RAG-система живёт в двух режимах: indexing (offline) и querying (online)
- Indexing pipeline: load → chunk → embed → store
- Query pipeline: embed query → retrieve → augment prompt → generate
- Naive RAG реально работает для простых задач

Naive RAG

Что мы узнали

- Embedding-модель должна быть одна и та же на индексации и на поиске
- Chunk — атомарная единица retrieval-a; от качества chunking-a зависит очень многое
- Retriever и Generator — независимые компоненты, оцениваются и улучшаются по отдельности
- "Garbage in — garbage out": плохой retrieval → плохой ответ, никакая LLM не спасёт

Chunking и подготовка данных

Почему Chunking — сложно?

- Chunk — это атомарная единица retrieval-а: либо весь чанк попадёт в контекст, либо не попадет
- Слишком маленький chunk → теряется контекст, ответ может быть в соседнем chunk-е
- Слишком большой chunk → "размывается" эмбеddинг, в контекст попадает много нерелевантного
- Нет универсального правильного размера — зависит от природы данных и типа запросов

Почему Chunking — сложно?

Конкурирующие требования

Желание	Тянет к	Конфликтует с
Точный retrieval	Маленькие чанки	Полнота контекста
Полный контекст	Большие чанки	Точность retrieval
Не разорвать смысл	Семантические границы	Равномерность размера
Уложиться в контекст LLM	Меньше размер и количество чанков	Полнота информации
Дешевизна embedding'a	Меньше чанков	Recall

Стратегия 1: Fixed-size chunking

- Разбиваем текст на куски фиксированного размера (по символам или токенам)
- Добавляем overlap для сохранения контекста на границах
- Это самая простая стратегия, baseline для всего остального
- Параметры:
 - *chunk_size* — типично 200-1000 токенов
 - *chunk_overlap* — типично 10-20% от *chunk_size*

Стратегия 1: Fixed-size chunking

Плюсы и минусы

- Плюсы:
 - Просто, быстро, предсказуемо
 - Работает для любого типа текста
 - Контролируемый размер чанков (важно для лимитов embedding-модели)
- Минусы:
 - Разрывает предложения посреди слова
 - Игнорирует структуру документа (заголовки, абзацы)
 - Может разделить связанные мысли (например, вопрос и ответ)

Стратегия 2: Recursive chunking

- Разбиваем по иерархии сепараторов: сначала по двойному переносу строки (абзацы), если кусок всё ещё большой — по одиночному переносу, потом по точкам, по пробелам
- Стараемся сохранить семантически связанные единицы вместе
- Как работает алгоритм:
 - Пробуем разбить по абзацам. Если получившиеся куски $\leq chunk_size$ — готово
 - Если кусок больше — пробуем разбить его дальше по ``n``
 - Рекурсивно спускаемся до тех пор, пока не уложимся в лимит

Стратегия 2: Recursive chunking

Плюсы и минусы

- Плюсы:
 - Учитываем структуру текста (абзацы, предложения)
 - По-прежнему достаточно простой
 - Хорошо работает для большинства natural language текстов
 - Baseline в индустрии
- Минусы:
 - Не понимает семантики (просто структура)
 - Для специфических форматов (код, markdown, HTML) нужны кастомные сепараторы

Стратегия 3: Document-aware chunking

- Учитываем структуру конкретного формата документа:
 - Для Markdown — режем по заголовкам («#», «##»)
 - Для HTML — по тегам секций (<section>, <div class="article">)
 - Для PDF — по логическим секциям (если структура распознаётся)
 - Для кода — по функциям, классам, импортам
 - Для юридических документов — по статьям и пунктам

Стратегия 3: Document-aware chunking

Плюсы и минусы

- Плюсы:
 - Chunk-и соответствуют логическим единицам документа
 - Метаданные структуры (заголовки) помогают на этапе retrieval и в промпте
 - Особенно хорошо для технической документации, юридических текстов, статей с явной структурой
- Минусы:
 - Требуется, чтобы документ имел чёткую структуру (плохо парсится "сырой" PDF без разметки)
 - Размер chunk-ов получается неравномерным
 - Нужно писать кастомные splitter-ы для разных форматов

Стратегия 4: Semantic chunking

- Используем эмбединги для определения, где граница между смысловыми блоками
- Алгоритм:
 - Разбиваем текст на предложения
 - Эмбедем каждое предложение
 - Считаем similarity между соседними предложениями
 - Где similarity резко падает → граница между chunk-ами

Стратегия 4: Semantic chunking

Плюсы и минусы

- Плюсы:
 - Chunk-и реально соответствуют смысловым блокам
 - Хорошо работает для текстов без явной структуры (статьи, эссе, транскрипты)
- Минусы:
 - Дороже: embedding-и на этапе indexing-a
 - Чувствительно к параметру порога близости
 - Не всегда работает лучше: иногда "резкая смена темы" — это не граница chunk-a, а просто естественный переход в одном смысловом блоке

Стратегия 5: Late chunking

- Классически: сначала разрезаем текст на chunk-и, потом эмбедем каждый отдельно
- Late chunking: сначала эмбедем весь длинный контекст одним проходом (используя long-context embedding model), потом нарезаем на чанки уже на уровне токен-эмбедингов и агрегируем (mean pooling)

Стратегия 5: Late chunking

- Что это даёт:
 - Каждый чанк помнит весь контекст документа в своём эмбединге
 - Решается классическая проблема: chunk «...он подписал указ...» — кто такой «он»?
- Ограничения:
 - Требуется long-context embedding model
 - Документ должен помещаться в контекст embedding-модели целиком (или по крупным частям)
 - Сложнее реализовать руками

Contextual Retrieval

- Подход, опубликованный Anthropic, дающий значительный прирост качества
- Идея: перед эмбедингом каждого chunk-а добавить к нему краткий контекст из всего документа, сгенерированный LLM
- Алгоритм:
 - Для каждого чанка прогоняем LLM-вызов с промптом: «Дан документ и фрагмент из него. Дай 1-2 предложения контекста, помогающего понять этот фрагмент в общей картине документа.»
 - Добавляем этот контекст как префикс к chunk-у
 - Эмбедем уже расширенный чанк

Contextual Retrieval

- Результаты Anthropic (на их бенчмарках):
 - Снижение failed retrievals на ~35%
 - В сочетании с reranking — снижение на ~67%
- Стоимость:
 - Дорого на indexing: один LLM-вызов на каждый чанк
 - Но благодаря prompt caching можно дешевле — документ кэшируется один раз, чанки запрашиваются с тем же кэшем

Метаданные: что хранить с каждым chunk-ом

- Помимо текста, с каждым чанком нужно хранить структурированные метаданные
- Метаданные используются для: фильтрации на retrieval, контекстуализации в промпте, audit trail, контроля доступа

Метаданные: что хранить с каждым chunk-ом

- Идентификаторы: id документа, id chunk-a
- Структурная позиция: заголовок документа, главы, секции, номер страницы
- Темпоральные данные: дата публикации документа, дата индексации документа, дата вступления в силу (например, для законов)
- Данные источника: url, тип источника
- Контроль доступа: публичный / приватный / роли доступа к chunk-у
- Семантические тэги: язык документа, освещаемые топики / темы

Специальные стратегии

Small-to-big retrieval

- Идея: эмбеддить маленькие чанки (для точности retrieval), но возвращать большие контексты вокруг них (для полноты)
- Реализация:
 - Документ режется на крупные секции (родительские chunk-и, ~2000 токенов)
 - Каждая секция режется на маленькие фрагменты (дочерние chunk-и, ~200 токенов)
 - В индекс добавляются эмбединги дочерних chunk-ов
 - При retrieval: находим релевантные дочерние chunk-и, возвращаем родительские метаданные

Специальные стратегии

Multi-vector retrieval

- Один чанк представляется несколькими векторами:
 - Вектор оригинального текста
 - Вектор LLM-сгенерированного summary
 - Векторы гипотетических вопросов, на которые этот chunk отвечает
 - Вектор краткого описания таблицы / картинки в chunk-е
- При поиске любой из этих векторов может смэтчиться с запросом
- Полезно: запрос пользователя часто формулируется как вопрос, а текст в базе — как утверждение

Как выбрать стратегию chunking для вашей задачи

Шаг 1. Какой формат документов?

- Markdown / HTML с разметкой → Document-aware (по заголовкам)
- Структурированные юридические / технические документы → Document-aware (по статьям/секциям)
- Код → Document-aware для конкретного языка
- Сырой текст без структуры (статьи, транскрипты) → Recursive или Semantic
- PDF плохого качества → сначала качественный парсинг, потом любая стратегия

Как выбрать стратегию chunking для вашей задачи

Шаг 2. Какие запросы пользователей?

- Короткие фактологические («когда вступил в силу закон X») → маленькие чанки (200-400 токенов)
- Развёрнутые объяснения («как работает процедура Y») → большие чанки (800-1500 токенов) или small-to-big
- Сравнительные («сравни X и Y») → нужны другие техники

Как выбрать стратегию chunking для вашей задачи

Шаг 3. Сколько фрагментов в типичном ответе?

- Один точный фрагмент → маленькие чанки
- Несколько связанных кусков → средние чанки + top-k побольше
- Целые длинные фрагменты → крупные чанки / small-to-big

Как выбрать стратегию chunking для вашей задачи

Шаг 4. Какой бюджет на индексацию?

- Низкий → recursive chunking, дешёвая embedding-модель
- Средний → document-aware + базовые метаданные
- Высокий → contextual retrieval + multi-vector

Как выбрать стратегию chunking для вашей задачи

Шаг 5. Что критично — точность или полнота?

- Точность (юридические, медицинские, финансовые вопросы) → маленькие чанки + строгие промпты
- Полнота (обзоры, аналитика) → большие чанки или small-to-big

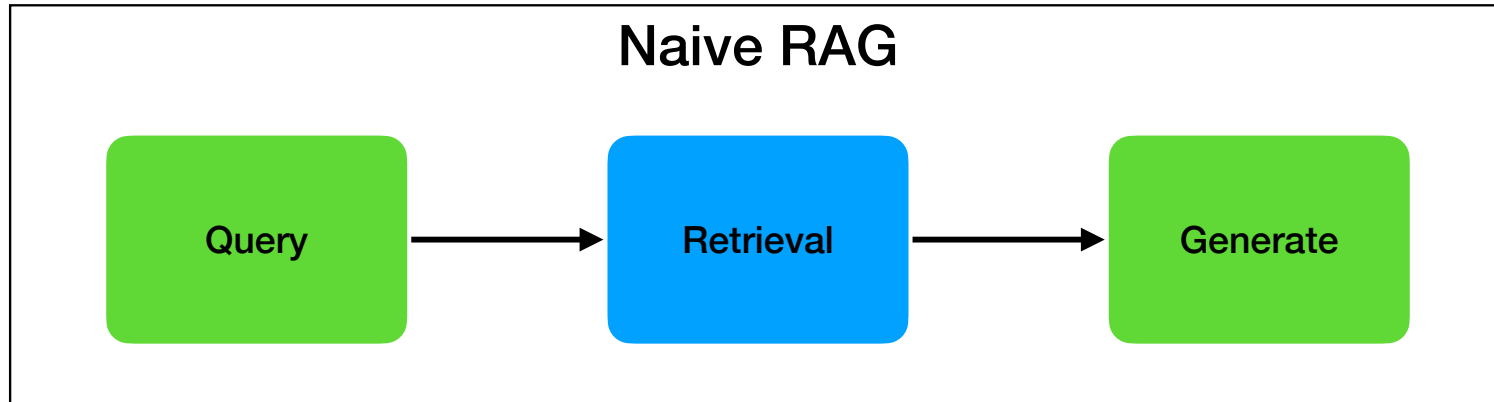
Chunking

Что мы узнали

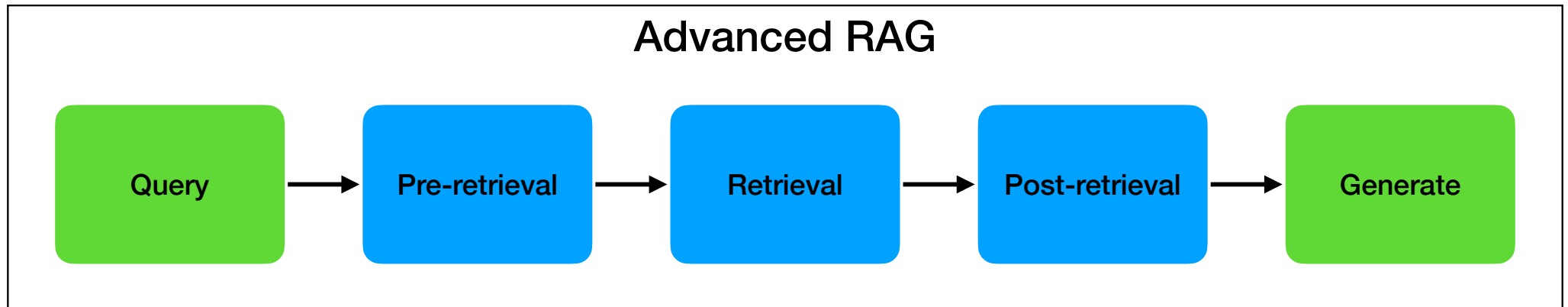
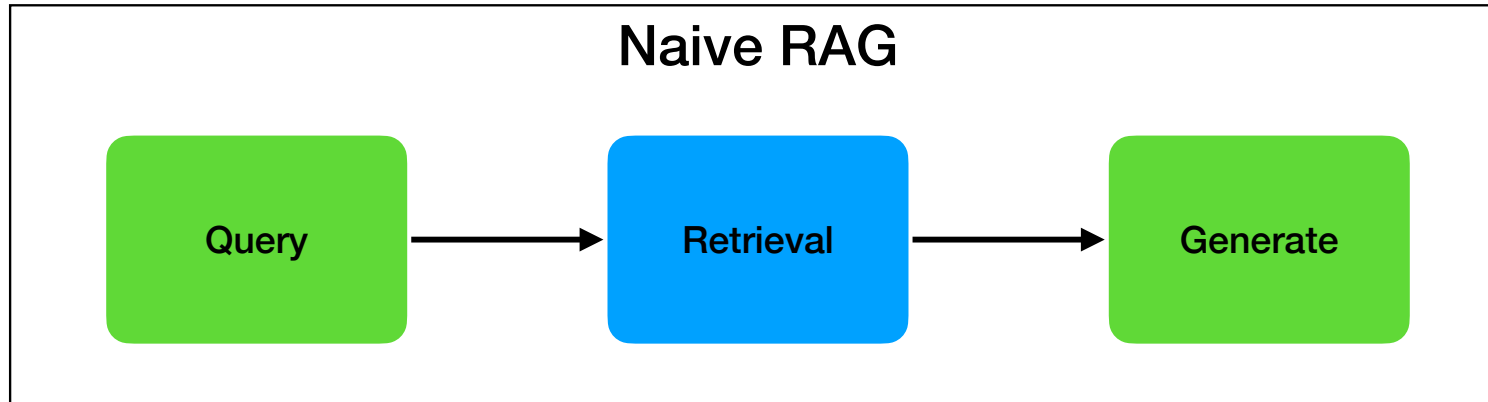
- Chunking — это компромисс между точностью и полнотой, нет универсального решения
- Стратегии по нарастанию сложности: fixed-size → recursive → document-aware → semantic → late chunking
- Contextual Retrieval — мощный приём с заметным приростом качества
- Метаданные критичны: для фильтрации, контекста в промпте, контроля доступа
- Small-to-big и multi-vector — продвинутые техники, когда нужно больше контроля

Продвинутые техники retrieval-a

Анатомия retrieval-пайплайна



Анатомия retrieval-пайплайна



Анатомия retrieval-пайплайна

Pre-retrieval — трансформации запроса

- Что хочет пользователь на самом деле?
- Как переформулировать запрос для лучшего поиска?
- Нужно ли разбить запрос на подзапросы?

Анатомия retrieval-пайплайна

Retrieval — собственно поиск

- По чему ищем (только dense? или гибридный поиск?)
- Сколько результатов берём (top-k)?
- Какие фильтры применяем?

Анатомия retrieval-пайплайна

Post-retrieval — обработка результатов

- Какие из найденных кандидатов реально релевантны?
- В каком порядке подавать их в контекст?
- Как сжать контекст, если кандидатов много?

Мотивация для всех pre-retrieval техник

Проблема vocabulary mismatch

- Запрос пользователя и текст документа говорят об одном и том же разными словами
- Классическая проблема классического IR (отсюда стемминг, лемматизация, query expansion)
- Не исчезает с переходом к dense retrieval — только меняет форму

Проблема vocabulary mismatch

Примеры

- Лексический mismatch:
 - Запрос: «как уволиться по собственному желанию»
 - Документ: «процедура расторжения трудового договора по инициативе работника»

Проблема vocabulary mismatch

Примеры

- Лексический mismatch:
 - Запрос: «как уволиться по собственному желанию»
 - Документ: «процедура расторжения трудового договора по инициативе работника»
- Структурный mismatch:
 - Запрос: короткий, в форме вопроса («Какая ставка НДС на лекарства?»)»
 - Документ: длинный, в форме утверждения («статья 164. Налоговые ставки...»)

Проблема vocabulary mismatch

Примеры

- Семантический mismatch (запрос слишком абстрактный):
 - Запрос: «оптимизация работы с памятью в Python»
 - Документ: содержит конкретные термины — «garbage collection», «weak references», ...

Проблема vocabulary mismatch

Примеры

- Семантический mismatch (запрос слишком абстрактный):
 - Запрос: «оптимизация работы с памятью в Python»
 - Документ: содержит конкретные термины — «garbage collection», «weak references», ...
- Многозначность:
 - Запрос: "как настроить ключ"
 - Без контекста: SSH-ключ? API-ключ? Газовый ключ?

Pre-retrieval

Query rewriting и expansion

- Используем LLM, чтобы переформулировать и/или расширить исходный запрос перед поиском
- Идея: запрос пользователя часто кривой, неполный, неоднозначный — попросим LLM сделать «хороший поисковый запрос»

Pre-retrieval

Query rewriting и expansion

- Базовый query rewriting:
 - Промпт: «Перефразируй этот вопрос пользователя в развёрнутый поисковый запрос»
 - Запрос: «ндс лекарств»
 - Переписанный: «ставка НДС на лекарственные средства и медицинские товары в РФ согласно Налоговому кодексу»
- Query expansion:
 - Добавляем синонимы и связанные термины
 - «уволиться» → «уволиться, расторжение трудового договора, увольнение по собственному желанию»

Pre-retrieval

Query rewriting и expansion

- Когда полезно:
 - Запросы пользователей короткие и неформальные (типичный chat-интерфейс)
 - Большая разница между языком запросов и языком документов (формальный язык vs разговорный язык)
- Когда вредно:
 - Запросы и так детальные — переписывание может «увести» смысл
 - Чувствительные домены, где переформулировка может сместить интент

Pre-retrieval

HyDE (Hypothetical Document Embeddings)

- Идея, опубликованная в 2022 году: вместо того чтобы искать документ по эмбедингу запроса, ищем по эмбедингу гипотетического документа
- Алгоритм:
 - Берём запрос пользователя
 - Просим LLM сгенерировать возможный ответ (даже если он будет неточным или галлюцинированным)
 - Эмбедем этот сгенерированный ответ
 - Ищем в индексе по эмбедингу гипотетического ответа

Pre-retrieval

HyDE (Hypothetical Document Embeddings)

- Почему это работает:
 - Эмбеддинг короткого вопроса и длинного документа — в разных областях embedding space
 - Эмбеддинг похожего по форме ответа намного ближе к реальным документам
 - Даже если LLM придумала неправильные факты — структура и терминология ответа правдоподобны, и эмбеддинг попадает в правильную семантическую область

Pre-retrieval

HyDE (Hypothetical Document Embeddings)

- Пример:
 - Запрос: «сколько лет хранить бухгалтерские документы»
 - Сгенерированный гипотетический ответ: «Согласно ФЗ-402 'О бухгалтерском учёте', первичные документы должны храниться не менее 5 лет после отчётного года. Налоговые документы — 4 года согласно НК РФ...»
 - Эмбеддинг этого текста ищет реальные нормативные документы намного эффективнее, чем эмбеддинг короткого запроса
- Минусы:
 - Дополнительный LLM-вызов на каждый запрос → латентность и стоимость
 - Если LLM хорошо знает домен — отлично, если плохо — может увести в неправильную область

Pre-retrieval

Multi-query и декомпозиция

- Multi-query retrieval:
 - Просим LLM сгенерировать несколько переформулировок одного запроса
 - Запускаем поиск для каждой
 - Объединяем результаты
- Декомпозиция:
 - Используется для сложных составных запросов
 - LLM разбивает запрос на атомарные подвопросы, каждый ищется отдельно

Pre-retrieval

Step-back prompting

- Просим LLM сформулировать более общий вопрос, контекст которого нужен для ответа на конкретный
- Запрос: «может ли ИП на УСН работать с НДС?»
- Step-back вопрос: «в чём отличия систем налогообложения УСН и ОСН?»
- Ищем по обоим, даём LLM богатый контекст

Retrieval

Hybrid search

- Комбинируем dense retrieval (semantic, эмбединги) и sparse retrieval (lexical, BM25)
- Каждый метод ловит то, что другой пропускает
- Когда dense retrieval лучше:
 - Семантические перефразы ("как уволиться" ↔ "процедура расторжения договора")
 - Запросы на естественном языке
 - Кросс-лингвистические сценарии (с мультязычными моделями)
- Когда sparse retrieval (BM25) лучше:
 - Точные термины, аббревиатуры, коды (ФЗ-152, ст. 164, INN)
 - Имена собственные, числа, названия продуктов
 - Out-of-domain слова, которых модель не видела при обучении эмбедингов
 - Профессиональный жаргон узких ниш

Retrieval

Reciprocal Rank Fusion (RRF)

- Это простой и эффективный способ объединить ранжированные списки

- Для каждого документа d : $score(d) = \sum_i \frac{1}{(k + rank_i(d))}$ по всем источникам i

- где k — константа, $rank_i(d)$ — ранг документа в списке i
- Документ, попавший в топ обоих списков, поднимается выше документов, попавших в топ только одного

Retrieval

Hybrid search

- На практике гибридных подход почти всегда работает лучше dense-only подхода
- Особенно силён для русского с обилием юридической, медицинской, технической терминологии
- Поддержка из коробки в Elasticsearch, Qdrant, etc.

Retrieval

Metadata filtering

- Используем метаданные чанков для фильтрации до или вместе с векторным поиском
- Сужает пространство поиска, повышает точность

Retrieval

Metadata filtering — Типичные сценарии

- Темпоральные фильтры:
 - «Что было в законе на момент 2023 года?» → фильтр «valid_at <= 2023-12-31»
 - «Покажи документы за последний месяц» → фильтр «created_at >= last_month»

Retrieval

Metadata filtering — Типичные сценарии

- Темпоральные фильтры:
 - «Что было в законе на момент 2023 года?» → фильтр «valid_at <= 2023-12-31»
 - «Покажи документы за последний месяц» → фильтр «created_at >= last_month»
- По типу документа:
 - «Найди в регламентах...» → фильтр «doc_type = «regulation»»
 - «Покажи только официальные ответы поддержки» → фильтр "source = «official_support»»

Retrieval

Metadata filtering — Типичные сценарии

- Структурные:
 - «Что в главе про налогообложение?» → фильтр «section = «Налогообложение»»
 - «Найди в FAQ» → фильтр «doc_type = «faq»»

Retrieval

Metadata filtering — Типичные сценарии

- Структурные:
 - «Что в главе про налогообложение?» → фильтр «section = «Налогообложение»»
 - «Найди в FAQ» → фильтр «doc_type = «faq»»
- Контроль доступа:
 - Пользователь Иванов делает запрос → фильтр «allowed_users contains «Ivanov»» или «access_level <= «internal»»
 - Это единственный надёжный способ обеспечить, что пользователь не получит данные, к которым у него нет доступа

Retrieval

Pre-filtering vs post-filtering

- Pre-filtering: сначала фильтр по метаданным, потом векторный поиск среди отфильтрованных.
 - Быстрее при сильной фильтрации, но может пропустить релевантное, если фильтр слишком узкий
- Post-filtering: векторный поиск, потом фильтрация.
 - Может вернуть меньше k результатов, если многие отсеялись
- Auto-filtering (LLM-driven):
 - LLM анализирует запрос пользователя и сам формирует фильтры
 - Запрос: «что говорил CEO в отчётах за прошлый год?»
 - LLM выдаёт: {"author": "CEO", "year": 2025, "doc_type": "report"}

Retrieval

ColBERT и late interaction

- Промежуточный подход между bi-encoder (dense retrieval) и cross-encoder — ColBERT (multi-vector encoder)
- Идея:
 - Bi-encoder: документ → один вектор. Запрос → один вектор. Сравнение: один dot product
 - Cross-encoder: документ + запрос → BERT → score. Более точно, но дорого
 - ColBERT: документ → много векторов (по одному на токен). Запрос → много векторов. Сравнение: для каждого токена запроса — max-similarity по токенам документа, потом сумма

Retrieval

ColBERT и late interaction

- Преимущества:
 - Захватывает более тонкие соответствия (разные части документа отвечают разным частям запроса)
 - Гораздо точнее bi-encoder, особенно на длинных и составных запросах
 - Можно индексировать заранее (в отличие от cross-encoder)
- Недостатки:
 - Индекс в 50-100 раз больше, чем для bi-encoder (хранится много векторов на документ)
 - Поиск медленнее
 - Не все vector DB хорошо поддерживают

Post-retrieval

Reranking с cross-encoder

- Стандартный двухэтапный подход в современном RAG: retrieve → rerank
- Сначала bi-encoder (быстро) находит ~50-100 кандидатов
- Потом cross-encoder (медленно, но точно) переранжирует и оставляет top-5 / top-10 документов
- Зачем?
 - Bi-encoder оптимизируется на recall: важно, чтобы правильный ответ попал в top-100
 - Cross-encoder оптимизируется на precision: правильный ответ должен оказаться в top-5
 - Cross-encoder видит запрос и документ вместе через self-attention, что даёт намного более точную оценку релевантности

Post-retrieval

Compression и extraction

- Найденные chunk-и часто содержат много лишнего: часть chunk-а релевантна, часть — нет
- Можно сжать контекст перед передачей в LLM, сохранив только важное
- Зачем сжимать?
 - Снижение стоимости (меньше входных токенов в LLM)
 - Часто — повышение качества: меньше шума в контексте → меньше lost-in-the-middle, меньше путаницы у модели

Post-retrieval

Compression и extraction — подходы

- LLM-extraction:
 - Для каждого чанка просим маленькую / быструю LLM выделить только релевантные предложения относительно запроса
 - Работает хорошо, но удваивает количество LLM-вызовов
- LLMlingua:
 - Использует маленькую модель для оценки «важности» каждого токена
 - Удаляет наименее важные токены, сохраняя смысл
 - Достигает сжатия в 2-5х с минимальной потерей качества
- Filtering:
 - Простой подход: с помощью cross-encoder отфильтровываем чанки со score ниже threshold
 - Если из top-20 после reranking-a ниже threshold оказались 15 — значит, у запроса просто мало релевантного материала, не надо засорять контекст

Post-retrieval

Reordering и lost-in-the-middle

- В 2023 в статье "Lost in the Middle" показали, что LLM плохо использует информацию из середины длинного контекста
- Лучше всего работает информация в начале и в конце промпта
- Это поведение сохраняется и в современных моделях, хотя и в меньшей степени
- Практическое следствие — стратегия reordering-a — после reranking упорядочиваем chunk-и в контексте так:
 - Самые релевантные → в начале и в конце
 - Менее релевантные → в середине
- На длинных контекстах (>10 chunk-ов) — заметный прирост точности ответов, на коротких контекстах (3-5 chunk-ов) — эффект минимальный

tldr

Проблема	Техника
Запросы кривые/короткие	Query rewriting
Сложные составные запросы	Декомпозиция / multi-query
Большой gap между языком запроса и документа	HyDE
Запросы с точными терминами/кодами	Hybrid (dense + BM25)
Большая база с разными правами/датами	Metadata filtering
Нужна максимальная точность top-k	Reranking
Шум в найденном контексте	Compression / filtering
Длинный контекст (>10 чанков)	Reordering

Продвинутые техники retrieval-a

Что мы узнали

- Retrieval — это не одна операция, а пайплайн из трёх фаз: pre-retrieval, retrieval, post-retrieval
 - Pre-retrieval: query rewriting, HyDE, multi-query, декомпозиция, step-back
 - Retrieval: hybrid search (dense + sparse), metadata filtering, ColBERT/late interaction
 - Post-retrieval: cross-encoder reranking, compression/extraction, lost-in-the-middle reordering
- Каждая техника решает конкретный класс проблем — не всегда нужно применять все

Оценка качества RAG-систем

Почему оценка RAG — сложна

- В классическом IR оценивали только retrieval: правильно ли найдены документы
- В классическом NLP оценивали только генерацию: BLEU, ROUGE, perplexity
- В RAG нужно оценивать обе подсистемы и их связку, причём:
 - Retrieval может быть хорошим, generation — плохим
 - Retrieval может быть плохим, но generation выкрутится из плохого контекста
 - Retrieval может быть хорошим, generation технически правильным, но ответ всё равно бесполезен (например, не отвечает на вопрос пользователя)

Почему оценка RAG — сложна

Дополнительные сложности

- Множественность правильных ответов: «ставка НДС на лекарства — 10%» и «медицинские товары облагаются НДС по льготной ставке 10%» — оба правильны, но текстово разные
- Открытый домен: невозможно создать ground truth для всевозможных запросов
- Динамичность: данные в индексе обновляются, «правильный» ответ сегодня и через месяц могут отличаться
- Субъективность качества: для одного пользователя ответ из двух предложений идеален, для другого — слишком краток

Метрики retrieval

- Recall@k — Доля релевантных документов, попавших в top-k результатов, среди всех релевантных документов
 - если правильный чанк не в top-k — LLM никак не может правильно ответить
- Precision@k — Доля релевантных среди top-k результатов
 - нерелевантные чанки в контексте → шум, риск галлюцинаций
- MRR — Средний обратный ранг первого релевантного документа
 - полезно, когда ожидается ровно один правильный ответ
- nDCG@k — Учитывает и порядок, и градации релевантности

Метрики retrieval

- Recall@k — Доля релевантных документов, попавших в top-k результатов, среди всех релевантных документов
 - если правильный чанк не в top-k — LLM никак не может правильно ответить
- Precision@k — Доля релевантных среди top-k результатов
 - нерелевантные чанки в контексте → шум, риск галлюцинаций
- MRR — Средний обратный ранг первого релевантного документа
 - полезно, когда ожидается ровно один правильный ответ
- nDCG@k — Учитывает и порядок, и градации релевантности
- Особенность для RAG:
 - Retrieval-метрики требуют ground truth: для каждого запроса — список релевантных чанков
 - Главная боль: где взять этот ground truth для своих данных?

Метрики generation

- Faithfulness — Соответствует ли ответ предоставленному контексту?
 - ответ может быть фактически верным, но если он не следует из контекста — это сигнал, что модель "вспоминает" из весов
- Answer Relevance — Отвечает ли ответ на заданный вопрос?
 - ответ может быть фактически правильным и опираться на контекст, но не отвечать на вопрос пользователя
- Context Precision — Действительно ли релевантные чанки оказались в начале выданного контекста?
- Context Recall — Покрывают ли найденные чанки всю информацию, нужную для ответа?
- Answer Correctness — Сравнение с эталонным ответом (если он есть)
 - может включать и фактическую правильность, и семантическую близость к эталону

LLM-as-a-judge

- Идея: использовать сильную LLM (GPT-5, Claude Opus) как «судью» для оценки ответов
- Промпт: «Дан вопрос, ответ системы и эталон. Оцени по шкале 1-5 по критериям А, В, С. Объясни оценку.»
- Альтернатива: "Дан вопрос и два ответа. Какой лучше?"
- Что мы оцениваем LLM-судьей в RAG:
 - Faithfulness: «Вот контекст, вот ответ. Все ли утверждения ответа следуют из контекста?»
 - Relevance: «Вот вопрос, вот ответ. Отвечает ли он на вопрос?»
 - Correctness: «Вот вопрос, вот эталон, вот ответ системы. Эквивалентны ли они по смыслу?»

Известные проблемы LLM-as-a-judge

- Position bias
 - При попарной оценке модель часто предпочитает первый из вариантов
 - Как избежать: запускать в обе стороны и усреднять
- Verbosity bias
 - Модель часто предпочитает более длинные ответы
 - Как избежать: явно инструктировать "длина не важна"
- Self-preference
 - GPT-5 склонна выше оценивать ответы, сгенерированные GPT-5
 - Как избежать: использовать судью, отличного от генератора
- Нестабильность
 - Один и тот же промпт может дать разные оценки при разных запусках
 - Как избежать: temperature=0, несколько прогонов с усреднением
- Calibration
 - LLM плохо различает близкие уровни качества
 - Как избежать: использовать попарное оценивание вместо абсолютных шкал

Где взять валидационный датасет?

- Проблема:
 - Чтобы посчитать большинство метрик, нужны пары "вопрос — правильный ответ" или "вопрос — релевантные чанки"
 - Размечать вручную 1000 примеров — недели работы доменных экспертов
 - В начале проекта реальных пользовательских запросов ещё нет
- Решение: сгенерировать синтетический датасет с помощью LLM

Где взять валидационный датасет?

- Базовый пайплайн генерации:
 - Берём chunk из вашей базы знаний
 - Промпт LLM: «На основе этого фрагмента сгенерируй вопрос, ответ на который содержится в этом фрагменте»
 - Сохраняем тройку (вопрос, chunk-источник, эталонный ответ)
 - Повторяем для разных chunk-ов
- Усложнения для реалистичности:
 - Разные типы вопросов
 - Разные уровни сложности
 - Реалистичный язык

Human evaluation

Когда применять?

- Критичные домены (медицина, право, финансы)
- Для калибровки LLM-судей: убедиться, что судья согласуется с человеком
- Периодический аудит качества
- Сложные качественные аспекты, которые автоматика плохо ловит: тон, политкорректность, культурная уместность

От RAG к агентам

От RAG к агентам

- Запросы, для которых статический RAG достаточен:
 - «Какова ставка НДС на медицинские товары?» — один retrieval, ответ в одном chunk-е
 - «Что такое ФЗ-152?» — один retrieval, ответ собирается из нескольких чанков одной темы

От RAG к агентам

Когда RAG-пайплайн не справляется?

- Многошаговые:
 - Пример: «Кто написал музыку к фильму, в котором играл главный режиссёр Брата 2?»
 - Нужно: найти режиссёра → найти его актёрский фильм → найти композитора этого фильма

От RAG к агентам

Когда RAG-пайплайн не справляется?

- Многошаговые:
 - Пример: «Кто написал музыку к фильму, в котором играл главный режиссёр Брата 2?»
 - Нужно: найти режиссёра → найти его актёрский фильм → найти композитора этого фильма
- Сравнительные:
 - Пример: «Сравни условия УСН 6% и УСН 15% для ИП с выручкой 50 млн в 2025 году»
 - Нужны независимые retrieval'ы по двум режимам, потом синтез

От RAG к агентам

Когда RAG-пайплайн не справляется?

- Многошаговые:
 - Пример: «Кто написал музыку к фильму, в котором играл главный режиссёр Брата 2?»
 - Нужно: найти режиссёра → найти его актёрский фильм → найти композитора этого фильма
- Сравнительные:
 - Пример: «Сравни условия УСН 6% и УСН 15% для ИП с выручкой 50 млн в 2025 году»
 - Нужны независимые retrieval'ы по двум режимам, потом синтез
- Аналитические:
 - Пример: «Как изменилось регулирование ПДн с 2020 по 2025 год?»
 - Нужно найти версии ФЗ-152 разных лет, сравнить, обобщить

От RAG к агентам

Когда RAG-пайплайн не справляется?

- Вопросы с внешним контекстом:
 - Пример: «Можно ли применять УСН при моей текущей выручке?»
 - Нужно уточнить выручку у пользователя или из его профиля → потом искать нормы

От RAG к агентам

Когда RAG-пайплайн не справляется?

- Вопросы с внешним контекстом:
 - Пример: «Можно ли применять УСН при моей текущей выручке?»
 - Нужно уточнить выручку у пользователя или из его профиля → потом искать нормы
- Вопросы не требующие retrieval вообще:
 - Примеры: «Привет!», «Объясни своими словами, что такое НДС»
 - Тратить retrieval-вызов на это — лишняя стоимость

Почему классический RAG не справляется?

- Статический пайплайн относится одинаково ко всем запросам
- Но запросы фундаментально разные по требуемой стратегии решения
- Решение — дать системе выбирать стратегию в зависимости от запроса

LLM-агент

- LLM, которая в цикле использует инструменты (tools) и принимает решения о следующих действиях, называется агентом
- В отличие от обычного чата, агент не просто отвечает текстом — может вызывать функции, читать результаты, продолжать рассуждение
- Имеет какую-то форму состояния — память о предыдущих шагах в рамках задачи

Ключевые компоненты

- Инструменты (tools):
 - Функции, которые LLM может вызвать с параметрами
 - Получает результат вызова и продолжает работать
 - Примеры: `vector_search`, `web_search`, `calculator`, `sql_query`, `send_email`
- Function calling / tool use:
 - Механизм в современных LLM (GPT-5, Claude, Gemini) для структурированного вызова функций
 - Модель возвращает не текст, а JSON с именем функции и аргументами
- Цикл:
 - Модель → решение (вызвать tool или дать ответ) → вызов tool → результат → модель → ...
 - Заканчивается, когда модель решает, что задача решена, или достигнут лимит шагов
- Память:
 - Краткосрочная: контекст текущей задачи
 - Долгосрочная (опционально): база предыдущих диалогов, факты о пользователе

Простейший пример агента

- Запрос: «Какая погода в Москве и переведи в Фаренгейты?»
- Шаг 1: LLM вызывает `'get_weather(city=«Москва»')` → получает 15°C
- Шаг 2: LLM вызывает `'convert(value=15, from=«C», to=«F»')` → получает 59°F
- Шаг 3: LLM формулирует ответ «В Москве 15°C (59°F)»

ReAct: фундаментальная парадигма

Reasoning and Acting

- Чередование двух типов шагов: Reasoning (рассуждение) и Acting (действие)
- Модель явно «думает вслух» перед каждым действием
- Видит результат действия, рассуждает дальше
- «Размышление вслух» заставляет модель структурировать рассуждение
- Каждое действие мотивировано предыдущим размышлением — повышает осмысленность

ReAct: фундаментальная парадигма

Структура ReAct

Thought: Мне нужно найти ставку НДС для медицинских товаров.

Это специфическая информация, нужен поиск в нормативной базе.

Action: vector_search("ставка НДС медицинские товары")

Observation: [найден 3 чанка из НК РФ]

Thought: В первом чанке указана ставка 10% по статье 164.

Нужно проверить, относятся ли лекарства к этой категории.

Action: vector_search("перечень медицинских товаров по льготной ставке НДС")

Observation: [найден чанк с перечнем]

Thought: Лекарственные средства входят в перечень. Могу формулировать ответ.

Action: finish("Ставка НДС для лекарственных средств — 10%, согласно ст. 164 НК РФ.")

Tool use в современных LLM

Function calling

- Способность LLM возвращать структурированный вызов функции вместо текста
- Разработчик описывает доступные функции (имя, параметры, описание) в виде JSON-схемы
- Модель сама решает, когда какую функцию вызвать

Tool use в современных LLM

Function calling — современные возможности

- Parallel tool calling:
 - Модель может вызвать несколько функций одновременно в одном шаге
 - Полезно для независимых retrieval-ов: «сравни X и Y» → одновременный поиск по обоим
- Structured outputs:
 - Гарантия, что вывод соответствует JSON-схеме
 - Полезно для классификации запросов, извлечения сущностей
- Streaming tool calls:
 - Получение вызова функции по мере генерации, начало выполнения раньше окончания генерации

Self-RAG

Одна из первых адаптивных архитектур RAG

- Модель в процессе генерации сама решает: нужен ли retrieval сейчас, и оценивает качество найденного
- Идея: вместо того чтобы всегда ретривить, модель генерирует специальные reflection-токены:
 - [Retrieve] — нужен поиск
 - [No Retrieve] — поиск не нужен, отвечаю из параметрической памяти
 - [Relevant] / [Irrelevant] — оценка найденного
 - [Supported] / [Partially Supported] / [No Support] — оценка, насколько ответ опирается на контекст

Self-RAG

Пайплайн

- Модель начинает генерировать ответ
- В нужный момент генерирует [Retrieve] → запускается поиск
- Получает результаты, оценивает релевантность токеном [Relevant] или [Irrelevant]
- Если релевантно — продолжает генерацию, опираясь на контекст
- После генерации помечает, насколько утверждение поддержано контекстом

Self-RAG

- Преимущества:
 - Адаптивность: тратит retrieval только когда он реально нужен
 - Прозрачность: метаданные о решениях встроены в ответ
 - Возможность fallback: если retrieval нерелевантен, модель явно это видит
- Ограничения:
 - В оригинале требовал fine-tuning модели на специальных reflection-токенах
 - В современных LLM идея реализуется через промптинг и tool definitions, без fine-tuning
 - Концепция важнее конкретной реализации

Corrective RAG

Что делать, если найденное — плохое

- Идея: после retrieval добавляется шаг оценки качества найденного
- В зависимости от оценки — разные стратегии действия
- Если локальной базы знание не хватает — ищем в интернете

Corrective RAG

Алгоритм

- Запрос → стандартный retrieval из локальной базы
- Лёгкая evaluator-модель оценивает каждый chunk
- По результатам — три ветки:
 - **Confident** (нашли хорошее):
 - Используем найденное как есть
 - Стандартная RAG-генерация
 - **Incorrect** (всё плохое):
 - Локальная база не помогла, делаем ищем в интернете
 - Используем результаты поиска в интернете как контекст
 - **Ambiguous** (часть хорошее, часть плохое):
 - Комбинируем: используем хорошие chunk-и + поиск в интернете
 - Можем дополнительно сжать/отфильтровать

Corrective RAG

- Преимущества:
 - Защита от случая, когда локальной базы недостаточно
 - Не требует верить retrieval-у — есть явная проверка
 - Поиск в интернете расширяет возможности системы
- Ограничения:
 - Поиск в интернете добавляет стоимость
 - Требуется доступа в интернет (что не всегда возможно)
 - Качество evaluator-модели критично
- Применимость:
 - Системы, где база знаний не покрывает все возможные запросы
 - Customer support, где могут спросить про вообще что угодно
 - Не подходит для строго сценариев, где поиск в интернете недопустим

Adaptive RAG

Выбор стратегии в зависимости от типа запроса

- Идея: классифицировать запрос по сложности и применить соответствующую стратегию
- Три категории запросов:
 - Simple (No retrieval) — На запрос можно ответить из общих знаний LLM
 - Single-step — Стандартный фактологический запрос → один retrieval достаточен
 - Multi-step — Сложный, требующий нескольких retrieval-ов и рассуждений → agentic loop с декомпозицией и итеративным retrieval

Adaptive RAG

Как реализуется маршрутизация

- Легковесный классификатор:
 - Маленькая модель (или сам LLM с быстрым промптом) классифицирует запрос
 - Запрос → классификатор → метка → выбор пайплайна
- Классификация той же моделью:
 - LLM сама помечает запрос как часть рассуждения
 - Промпт: «Сначала определи: нужен ли вообще поиск? Если да — простой запрос или сложный?»

Adaptive RAG

- Преимущества:
 - Экономия: простые запросы не дёргают retrieval и тяжёлый пайплайн
 - Скорость: ответы на простые вопросы — мгновенные
 - Качество: сложные запросы получают полноценную обработку
- Подводные камни:
 - Классификатор может ошибиться: пометить сложный запрос как simple → провал
 - Особенно опасно — пометить требующий retrieval как simple → галлюцинация
 - Безопасный fallback: при сомнениях — делать retrieval
- Связь с практикой:
 - Большинство production-RAG в b2c так или иначе делают маршрутизацию
 - Часто это первостепенная защита от лишних API-вызовов

Agentic RAG

- Полноценный LLM-агент с набором инструментов
- Retrieval — один из tool-ов, наравне с web-search, calculator, SQL-query, API-вызовами и т.д.
- Агент сам планирует и выполняет последовательность действий

Agentic RAG

Типичный набор tool-ов

- Retrieval tools:
 - *vector_search(query, collection)* — поиск в одной из специализированных коллекций
- External tools:
 - *web_search(query)* — поиск в интернете
 - *fetch_url(url)* — загрузка конкретной страницы
- Computational tools:
 - *code_interpreter(code)* — выполнение кода для анализа данных
- Database tools:
 - *sql_query(query, db)* — структурированные данные
- Domain-specific:
 - Например, для CRM: *get_order_history(customer_id)*

Agentic RAG

Пример сценария

- Запрос: «Оцени, можно ли применить УСН для нашей компании в следующем году»
- Агент:
 - *lookup_customer(current_user)* → получает данные компании
 - *sql_query("SELECT revenue FROM finances WHERE company_id=...")* → текущая выручка
 - *vector_search("лимиты УСН 2026")* → актуальные нормы
 - *calculator(...)* → прогноз выручки
- Синтез: «Ваша прогнозируемая выручка X превышает лимит УСН Y, поэтому...»

Agentic RAG

- Преимущества:
 - Гибкость: один агент покрывает широкий спектр задач
 - Композиционность: легко добавлять новые tools
 - Естественная декомпозиция: агент сам разбивает сложную задачу на шаги
- Сложности:
 - Латентность: каждый tool-call — отдельная операция, может быть много шагов
 - Стоимость: каждый шаг = LLM-вызов
 - Надёжность: агент может уйти в бесконечный цикл, неправильно использовать tool, забыть результат
 - Отладка: длинные trace-ы работы агента сложно анализировать

Память и состояние в агентах

Виды памяти в агентных системах

- Short-term (контекст задачи):
 - все шаги текущего цикла: thought, action, observation
 - накапливается в промпте по мере выполнения
 - очищается между разными задачами

Память и состояние в агентах

Виды памяти в агентных системах

- Short-term (контекст задачи):
 - все шаги текущего цикла: thought, action, observation
 - накапливается в промпте по мере выполнения
 - очищается между разными задачами
- Conversational (история диалога):
 - предыдущие сообщения в текущей сессии
 - часто реализуется через суммаризацию при превышении лимита токенов

Память и состояние в агентах

Виды памяти в агентных системах

- Long-term (персистентная):
 - факты о пользователе: имя, профессия, предпочтения, история запросов
 - хранится во внешнем хранилище (БД, vector store)
 - LLM получает релевантные факты через специальный tool типа *recall_user_context()*

Память и состояние в агентах

Виды памяти в агентных системах

- Long-term (персистентная):
 - факты о пользователе: имя, профессия, предпочтения, история запросов
 - хранится во внешнем хранилище (БД, vector store)
 - LLM получает релевантные факты через специальный tool типа *recall_user_context()*
- Episodic (опыт):
 - воспоминания о прошлых решённых задачах
 - в прошлый раз похожий запрос я решил так — попробую аналогично

Архитектуры памяти

- Buffer (наивная):
 - Просто список последних N сообщений
 - Простая, но не масштабируется
- Summarization:
 - Старая часть истории заменяется на саммари
 - Сохраняет основное, экономит токены

Архитектуры памяти

- Vector memory:
 - История диалога индексируется в векторное хранилище
 - При новом запросе ищутся релевантные предыдущие сообщения
 - Это уже "RAG over conversation history"
- Reflection и self-improvement:
 - Агент после выполнения задачи рефлексит: что получилось, что нет, как улучшить
 - Reflection сохраняется в long-term memory
 - Используется при будущих похожих задачах

Multi-agent системы

- Декомпозиция задачи на роли
- Каждый агент специализирован: разный системный промпт, набор tool-ов, иногда разная LLM
- Агенты взаимодействуют через сообщения или общее состояние

Multi-agent системы

Типичные роли в мультиагентной RAG-системе

- Planner (планировщик) — разбивает сложный запрос на подзадачи
- Researcher (поисковик) — ищет максимум релевантной информации, используя разные retrieval-ы, поиск в интернете
- Critic (критик) — оценивает промежуточный результат, имеет возможность инициировать дополнительный retrieval
- Writer (писатель) — componeет финальный ответ

Зрелость multi-agent системы

- Что работает хорошо:
 - Простые роли с чёткими интерфейсами (researcher + writer)
 - Системы с критическим шагом (например, retrieval) и финализирующим writer-ом
- Что работает плохо:
 - Сложные многоагентные дискуссии "до консенсуса" — часто уходят в шум
 - Агенты, дублирующие работу друг друга
 - Системы из 5+ агентов с гибким взаимодействием — сложно отлаживать, дорого
- Вывод: не нужно делать multi-agent ради multi-agent

Deep Research

Production-кейс поиска с агентами

- Агент, который итеративно исследует тему: ищет, читает, рассуждает, ищет дальше, синтезирует
- Цель — выдать полноценный аналитический отчёт по сложному запросу за минуты-десятки минут
- Это не «ответ на вопрос», это «исследование по теме»
- Известные реализации:
 - OpenAI Deep Research
 - Anthropic Research
 - Google Gemini Deep Research

Упрощенная архитектура Deep Research

- Planning phase: агент формулирует план исследования — какие подвопросы изучить
- Iterative research loop:
 - Для каждого подвопроса — серия поисков
 - Чтение и извлечение фактов
 - Оценка покрытия: что узнали, чего не хватает
 - Уточняющие поиски
- Synthesis phase: компоновка финального отчёта с цитатами

Что отличает Deep Research от обычного RAG

- Десятки и сотни retrieval-шагов вместо одного
- Активное планирование и переоценка плана по ходу работы
- Модель держит контекст всего исследования
- Время работы: минуты, а не секунды
- Стоимость: десятки центов до доллара за один запрос

От RAG к агентам

Когда агенты оправданы

- Сложные аналитические задачи, которые невозможно решить за один шаг
- Большое разнообразие типов запросов
- Сценарии, где пользователь готов ждать
- Высокая ценность результата оправдывает стоимость

От RAG к агентам

Когда лучше остановиться на классическом RAG

- Простые фактологические запросы
- Высокий QPS, низкая ценность каждого ответа
- Жёсткие требования по времени ответа (<1 сек)
- Зарегулированный домен с требованием воспроизводимости

Тренды 2025-2026

1. Long context vs RAG

- Модели с контекстом 1M-10M токенов делают long-context подход конкурентным для средних баз (помещаем всю базу в контекст LLM)
- Но RAG не умирает: для больших баз и приватных данных остаётся доминирующим
- Тренд — гибридные подходы, где retrieval отдаёт большие куски в длинный контекст
- Prompt caching делает это экономически оправданным

2. Multimodal RAG

- Retrieval не только по тексту, но и по изображениям, диаграммам, таблицам
- CLIP-style эмбединги для картинок, специализированные модели для таблиц
- Особенно важно для научных статей, технической документации с диаграммами, презентаций
- В последние годы появились зрелые мультимодальные эмбединг-модели

3. GraphRAG и structured retrieval

- Идея: строим граф сущностей и связей из корпуса, ищем не чанки, а подграфы
- Хорошо работает для аналитических, сравнительных запросов
- Дорого в построении, но даёт качественно новые возможности

4. Agentic-first архитектуры

- Сдвиг от «RAG как продукт» к «агент с retrieval-ом как одним из инструментов»
- Появление production Deep Research продуктов

5. Ренессанс late interaction и multi-vector retrieval

- ColBERT-подходы стали практичнее благодаря оптимизациям
- Особенно сильны для длинных и составных запросов
- Современные векторные БД нативно поддерживают

6. Contextual Retrieval и продвинутые стратегии

- Anthropic Contextual Retrieval — огромный прирост качества
- Late chunking, structure-aware chunking
- Общий тренд: больше интеллекта в preprocessing, меньше полагания на волшебство эмбеддингов

7. Specialized embeddings

- Модели под конкретные домены и задачи:
 - Code embeddings
 - Biomedical embeddings
 - Legal embeddings

8. Reasoning + retrieval интеграция

- Модели с reasoning меняют подход к агентам
- «Retrieve then generate» → «Think then retrieve»
- Новые архитектуры: модель в процессе reasoning сама решает, когда остановиться и поискать

9. Персонализация RAG

- Память о пользователе как часть контекста
- Персонализированный ranking на основе истории
- Баланс с приватностью ПДн — активная область исследований

10. Открытые проблемы

- Галлюцинации даже при хорошем retrieval — полностью не решены
- Оценка агентных систем с длинными диалогами / траекториями исследований
- Безопасность LLM-агентов с доступом к действиям
- Экономика масштабных систем: как делать RAG дешёвым

RAG и Агенты

Андросов Дмитрий, 04.05.2026, AI Masters